Selecting Reliable Reasoning with Arithmetic and Candidate Attention

DASE7506 Advanced Machine Learning Methods | MP2 | 31 October 2026

Abstract

I study whether arithmetic evidence and candidate interactions can improve selection among four fixed mathematical solutions under a small deployment budget. The submitted method combines a multi-view ranking and tree branch with two compact self-attention families, then averages scores for numerically equivalent answers. Arithmetic consistency, question context and candidate interactions provide complementary evidence for a compact learned verifier. The frozen model selects correct final answers on **402 of 467 test questions, or 86.0814%**, with identical predictions on two hosts. Complete CPU prediction and independent scoring take 8.893 seconds on an AMD EPYC 9354 using four numerical threads; inference files total 6.5046 MiB. Validation-only component deletions support the combination, while failure cases show that arithmetic features do not guarantee semantic correctness. Validation was used adaptively. The test result is benchmark reproduction rather than a blind evaluation, and the specified reference Xeon has not been measured.

1 Task and evaluation setting

The task is answer selection, rather than generating or repairing an answer. Each question has four supplied reasoning traces and a fixed 1024-dimensional embedding for each trace. I return one candidate index. The official parser maps equivalent fractions and decimals to the same numeric answer; the scorer checks this final answer, not the validity of every reasoning step. The data derive from GSM8K [1], but all experiments here use the course bank and its fixed representations.

Table 1. Official splits and their experimental roles.

Split	Questions	Informative	Use
TRAIN	4,755	2,114	Fit parameters and statistics
Validation	798	355	Development and fusion selection
TEST	467	467	Frozen scoring and reproduction

An informative group contains at least two distinct valid parsed answers. The remaining 443 validation groups have one shared valid answer, so their inclusion makes overall validation accuracy easier than the all-informative test bank. I therefore report both overall and informative validation accuracy. Multiple correct candidates are allowed; labels are obtained only from the corresponding permitted reference split.

The research question is whether complementary evidence improves selection beyond answer frequency and individual learned components while keeping complete CPU evaluation below the time and asset limits. I evaluate the implementation through individual components, fixed-weight ablations and cross-host reproduction.

2 Pipeline and evidence construction

Figure 1 separates fitting from deployment. All learned normalizers, projections, lexical statistics and model parameters come from TRAIN. Validation chooses a frozen fusion configuration. During inference the method reads only the current question, candidates, supplied embeddings and checkpoint.

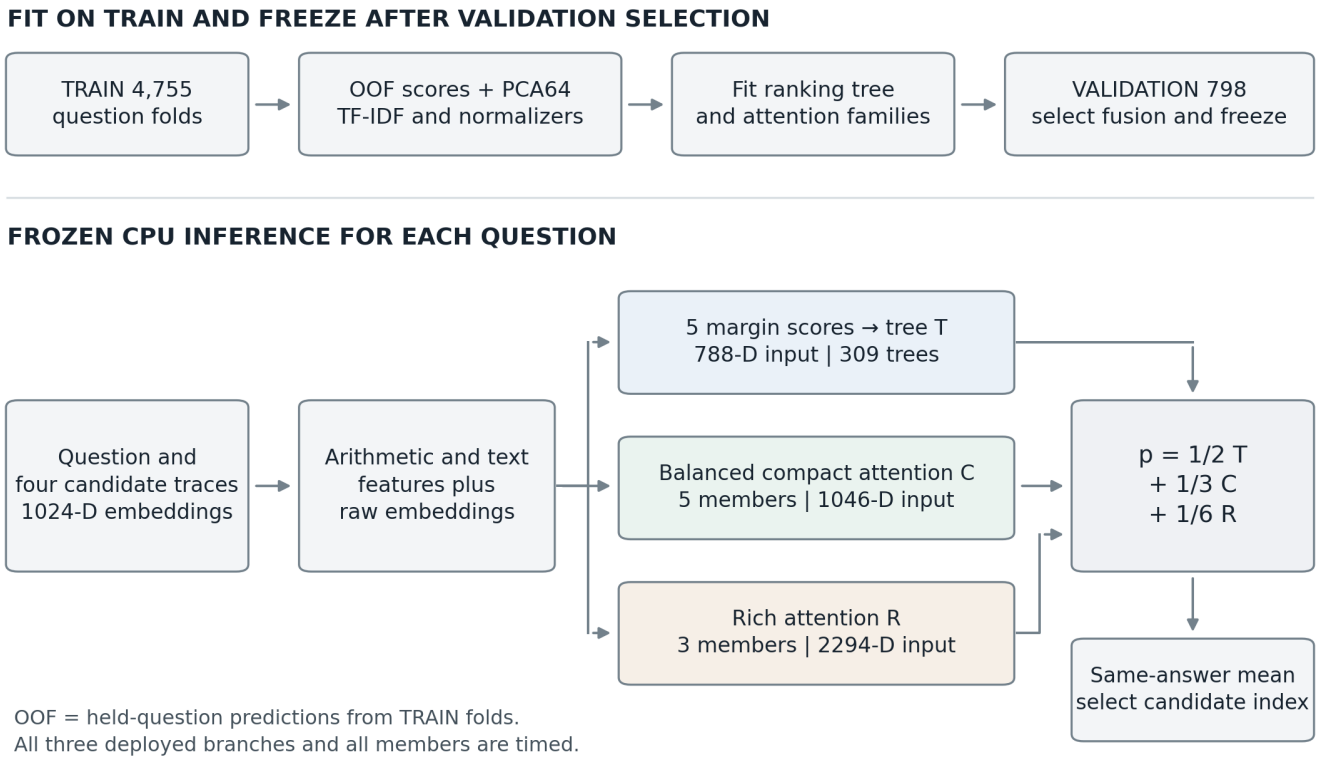


Figure 1. The submitted pipeline. OOF scores are predictions for TRAIN questions held out of their base-model fit. Only the selected Logloss tree, five balanced compact attention members and three rich attention members are deployed; the five linear margins are inputs to the tree.

Table 2. Feature families and tree input sizes.

Evidence	Dimension	Purpose
Rich candidate features	207	Format, answer votes, arithmetic and relative evidence
Compact candidate features	22	Low-dimensional input for compact attention
Repair features	39	Numeric error propagation and answer support
Quantity and schema features	126 + 126	Compatibility with question quantities and forms
Tree inputs	619 / 788	OOF, PCA, quantity and extended evidence

The arithmetic module parses bounded numeric expressions and evaluates them with exact fractions. It checks explicit equalities and follows repeated erroneous numbers through later calculations. Number coverage, simple algebraic schemas and repair support become features, rather than commands to override the model. Matching repeated numbers is heuristic: equal values can have different roles in a word problem.

The 619-dimensional table contains 207 rich features, 30 base-score features, 256 PCA-derived context features and 126 quantity features. The extended table adds 126 schema, four lexical and 39 repair columns, giving 788. The tree combines these semantic, lexical and numeric signals in a shared per-candidate feature space.

3 Training objectives and model selection

Five linear rankers use the original candidate embedding, its question context, or its same-answer cluster context, together with rich features. Two views also use derived groups with one positive and three negatives, or three positives and one negative. Derived rows remain attached to their original question. Each fitting question has equal total pair mass; duplicates of the same answer do not multiply that answer cluster's weight.

Five stratified question folds (seed 7526) produce base and lexical OOF evidence for stacked generalization [5]. Normalizers are estimated from fitting questions only. PCA64 and the final TF-IDF statistics are fitted on full TRAIN. The Logloss CatBoost head [4] uses a TRAIN-only 80/20 informative-question holdout, seed 7546, to choose iterations, then refits on all 2,114 informative TRAIN questions. The deployed head has 309 depth-six trees.

3.1 Joint candidate scorers

The attention scorers compare the four candidates jointly using pre-normalized self-attention [2], without positional embeddings. This is related to attention-based set modelling [3], but does not implement inducing-point attention or a Set Transformer decoder. The compact input is 1024 raw coordinates plus 22 features. The rich input adds a 1024-dimensional question residual, 207 features and 39 repair features, totalling 2294.

Table 3. Neural training configurations. LR is the AdamW learning rate.

Family	Width / layers	Members	Epochs	LR / decay
Compact balanced	128 / 2	5	15	0.0001 / 0.001
Rich balanced	96 / 1	3	1	0.0003 / 0.08

Every scorer has four attention heads and an expansion factor of two. Compact models use GELU, dropout 0.2 and final layer normalization; rich models use ReLU, dropout 0.3 and a zero-initialized linear skip. Compact members fit four fifths of TRAIN using a seed-13 question permutation and seed 37 initialization. Rich members fit all TRAIN with seeds 7506, 7507 and 7508. AdamW uses batches of at most 64 and gradient clipping at 1.0.

For scores s and sets V of valid candidates and P of correct candidates, the multi-positive mass loss is

$$L_{mass} = \log \sum_{j \in V} \exp(s_j) - \log \sum_{j \in P} \exp(s_j).$$

Compact training uses (1 + hard) × (BCE + 0.5 L_{mass}). Rich training uses 0.5 L_{pair} + 0.5 L_{mass} + 0.15 BCE, where L_{pair} averages softplus($s_{negative} - s_{positive}$) over eligible pairs. These losses preserve multiple correct answers.

3.2 Frozen fusion

A prescribed 309-configuration validation grid compares weights in sixths and pooling strengths 0, 0.6 and 1. The tree weight total is restricted to one third through two thirds, with a nonzero compact-family contribution. Selection prioritizes correct questions, then correct choices on wrong-majority single-correct groups, then candidate Brier loss. The selected probabilities are p = 0.5 T + (1/3) C + (1/6) R. Within each parsed-answer group they are averaged; the highest group score determines the first candidate with that answer. FP16-exported parameters are used before validation selection.

4 Validation and formal test results

Table 4 compares answer-frequency and first-candidate baselines with the three deployed components and the frozen ensemble. All learned component rows use same-answer mean pooling. The 60 trap questions have exactly one correct candidate and a repeated wrong answer that wins the frequency baseline.

Table 4. Validation results. The last two columns separate informative questions and wrong-majority traps.

Method	Correct	Overall	Informative	Traps
Majority answer	688/798	86.22%	245/355	0/60
First candidate	624/798	78.20%	195/355	19/60
Logloss tree	751/798	94.11%	308/355	33/60
Balanced compact attention	739/798	92.61%	296/355	44/60
Rich attention	740/798	92.73%	297/355	47/60
Full frozen ensemble	758/798	94.99%	315/355	41/60

The ensemble gains 70 correct validation choices over majority voting, with 82 fixes and 12 regressions. On the informative subset it reaches 315/355, or 88.73%, compared with 245/355, or 69.01%, for majority. Relative to the Logloss tree it fixes nine choices and loses two, a net gain of seven. The rich family is strongest on the trap subset at 47/60, while the ensemble has 41/60: optimizing total accuracy does not maximize every subgroup.

4.1 Formal test scoring

The fusion configuration was selected on validation and frozen before test scoring. The official result is **402/467 = 86.0813704496788%**. The local workstation and new-ser produce identical IDs and all 467 choices, and the standalone release reproduces them. Component and ablation TEST cells are not evaluated. The 94.99% validation and 86.08% test percentages have different question compositions and should not be compared as if drawn from the same mix.

Fitting uses TRAIN, selection uses validation, and TEST is used for frozen scoring and reproduction. The test result should be interpreted as benchmark reproduction rather than an independent blind evaluation. No TEST item-level correctness analysis is performed.

5 Controlled validation diagnostics

These diagnostics were added after final test scoring to explain the submitted frozen model. They do not change it. Deleting a family renormalizes the remaining weights without a new weight search or retraining. The pooling ablation retains all probabilities and weights.

Table 5. Fixed component and pooling ablations. Differences are correct questions relative to 758/798.

Change	Validation	Difference	Traps
Remove Logloss tree	744/798	-14	46/60
Remove balanced compact attention	753/798	-5	35/60
Remove rich attention	755/798	-3	38/60
Remove same-answer pooling	753/798	-5	34/60

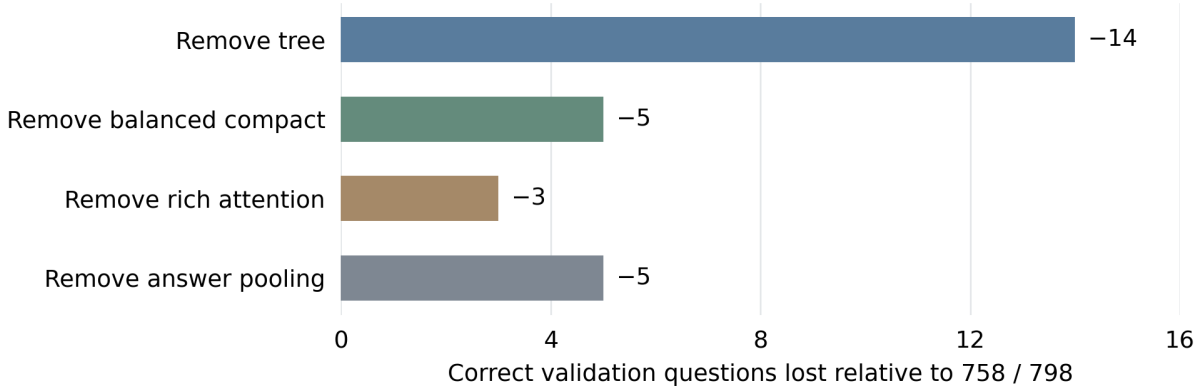


Figure 2. Losses in correct validation choices after each fixed change. The component-deletion results include renormalization, not reoptimized reduced ensembles.

The tree contributes the largest aggregate benefit: removing it loses 14 correct choices. Removing the balanced compact and rich families loses five and three. Pooling adds a net five choices, consistent with reducing reliance on one unusually high-scoring trace for a repeated answer.

These are development-set diagnostics, not independent confirmation. Deleting a component also changes score calibration and its normalized mixture weights. The results do not identify the isolated causal benefit of each feature or normalization choice. Those details would need matched refits with a separately specified validation protocol.

6 Validation examples and remaining failures

Both examples below are drawn from validation. They illustrate behaviour rather than estimate how often each failure type occurs; exact validation IDs, traces and probabilities are included in the report data.

6.1 Pooling can overcome a confident wrong trace

A show has ten seasons, with twenty episodes per season for the first half and twenty-five for the second half. The correct total is $5 \times 20 + 5 \times 25 = 225$. Three candidates instead apply each episode count to all ten seasons and answer 450. Their individual multiplications are valid; the mistake is the interpretation of “half”.

Table 6. Candidate and answer-level scores for the show example.

Answer	Candidates	Highest candidate score	Mean answer score
225 correct	1	0.67229	0.67229
450 wrong	3	0.75817	0.66494

Selecting the highest candidate probability chooses 450, while answer pooling chooses 225. The answer-score gap is only 0.00735, so this success is also an example of a fragile decision. Arithmetic checking alone does not resolve the mistaken allocation of seasons.

6.2 Repeated arithmetic errors can still win

A population question requires $(15 \times 58{,}940 - 130{,}000)/2 = 377{,}050$. Three traces instead write $15 \times 58{,}940 = 883{,}100$, propagate the 1,000-person error and answer 376,550. The exact arithmetic extractor detects this multiplication discrepancy in all three wrong traces, but their mean score is 0.65004 versus 0.57592 for the correct trace. The ensemble still selects the wrong answer. Verification signals enter a learned score; they do not impose a formal correctness constraint.

6.3 Score gaps and uncertainty

Table 7. Validation accuracy by gap between the two highest answer scores.

Top two answer-score gap	Informative questions	Correct	Accuracy
Below 0.25	54	35	64.81%
At least 0.25	301	280	93.02%

Small gaps identify a difficult subset on validation, but they are not calibrated probabilities of correctness. The first example has a very small gap and is correct, while the population example is wrong at a gap of 0.07412. No threshold is used to change the frozen test decisions.

6.4 Scope of the evidence

The validation set has been reused across development. The tree iteration holdout uses a fixed OOF base table and is not fully nested cross-validation of the entire pipeline. Numeric matching can confuse repeated values with different roles and miss semantic constraints. Separate audit-bank generalization and runtime on the reference Xeon remain unmeasured.

7 Computation and resource declaration

I trained the submitted v73 on my local workstation with an Intel Core i9-14900KF, 62.58 GiB of OS-visible RAM and **one NVIDIA GeForce RTX 4090D** (24,564 MiB reported GPU memory, driver 580.173.02). CPU numerical libraries were configured for four threads. Neural members were fitted sequentially on that GPU; no multi-GPU training was used.

The recorded final v73 run took **212.74 seconds**, including feature extraction, fitting, validation and the prescribed fusion search. Peak PyTorch allocated GPU memory was 388.13 MiB; this counter excludes non-PyTorch allocations and is not total system GPU memory. The recorded runtime excludes environment setup, audits and report production.

7.1 Complete fresh-process CPU evaluation

Table 8. Complete official test timing with four numerical threads.

Host CPU	Prediction	Scoring	Total
Intel Core i9-14900KF	3.363 s	0.077 s	3.440 s
AMD EPYC 9354 new-ser	8.763 s	0.130 s	8.893 s

Inference uses CPU NumPy only with four numerical threads. The measurement starts before each fresh prediction/scoring process and includes imports, supplied-input loading, model loading, feature calculation and every deployed member. A separate validation process inspection observed a peak of four OS threads, no child workers and no loaded CUDA or Torch libraries. Parsing caches start empty in the fresh process and store only computations on current inputs.

The new-ser server has 125.77 GiB OS-visible RAM. Neither reported CPU is the specified Xeon Platinum 8457C, so the 30-second reference-machine qualification is not claimed as verified.

7.2 All deployed inference assets

Table 9. All files in the deployed checkpoint, without outer-package compression.

Checkpoint file	Bytes	MiB
compact_balanced.npz	4,044,398	3.8570
config.json	504	0.0005
rich.npz	1,845,262	1.7598
stack.npz	754,576	0.7196
tree_log.npz	175,780	0.1676
Total	6,820,520	6.5046

The NPZ entries are uncompressed. FP16 neural parameters are converted to FP32 for CPU computation; even counting all arrays after this expansion gives 11.9954 MiB, below 16 MiB before the small container/configuration overhead. All deployed normalizers, projections, lexical coefficients, configuration and ensemble members are included. Training-only diagnostic caches are not read by inference. No external dataset, additional pretrained weights, evaluation-time generation, network API or stored-answer lookup is used.

8 Reproduction and related work

The release keeps the official data, parser and scorer unchanged. The entry point is `code/submission_v73.py`; the matching assets are in `code/artifacts/v73/checkpoint`. Frozen hashes, the before-test selection record, same-host remote receipt, official score JSON and cross-host choice comparisons are included. Source and asset hashes verify the submitted implementation.

8.1 Reproduction commands

From `code/`, use Python 3.12 and the pinned NumPy-only inference requirements. The verifier sets the four-thread limit and common numerical backend before launching the official tools. Output directories must be new.

```
python -m pip install -r requirements-inference-v73.txt
python -m reference_rebuild_v73.verify --run artifacts/v73 \
  --split test --output runs/reproduced-test
python report_analysis.py --output-dir ../report/recomputed-data
```

Training uses a separate Python 3.13 CUDA environment with pinned NumPy, SciPy, scikit-learn, CatBoost and PyTorch versions. The supplied training script needs no previous checkpoint or cache. To replay the full training audit, use the newly generated training output, since bulky search caches are omitted from the submission package.

```
python -m pip install -r requirements-train-v73.txt
CUDA_VISIBLE_DEVICES=0 python -m reference_rebuild_v73.train \
  --output artifacts/retrained-v73
python -m unittest test_reference_rebuild_v73 -v
```

9 unit tests pass. Normalization and validation-component replay errors were zero; base OOF probability differences were below 3×10^{-8}. Sampled unknown IDs, candidate/question permutations, single-question calls and blocked reference I/O passed. Cross-hardware retraining is not promised to be bitwise identical; frozen assets reproduce the submitted score.

8.2 Related work

Verifier-based selection [1] provides an outcome-supervision perspective for scoring the four fixed candidates. Candidate interactions use self-attention [2] over a set of solutions [3]. The tabular branch uses CatBoost [4] for nonlinear feature interactions. Held-question base predictions provide inputs to its meta model, following the principle of stacked generalization [5].

8.3 AI assistance

OpenAI Codex substantially assisted implementation, experiments, diagnostics, figures and report drafting. Saved experiment records and frozen assets accompany the code for reproduction.

References

[1] K. Cobbe et al. Training Verifiers to Solve Math Word Problems. arXiv:2110.14168, 2021. https://arxiv.org/abs/2110.14168

[2] A. Vaswani et al. Attention Is All You Need. Advances in Neural Information Processing Systems 30, 2017. https://arxiv.org/abs/1706.03762

[3] J. Lee et al. Set Transformer: A Framework for Attention-based Permutation-Invariant Neural Networks. ICML, PMLR 97, pp. 3744-3753, 2019. https://proceedings.mlr.press/v97/lee19d.html

[4] L. Prokhorenkova et al. CatBoost: unbiased boosting with categorical features. Advances in Neural Information Processing Systems 31, 2018. https://arxiv.org/abs/1706.09516

[5] D. H. Wolpert. Stacked generalization. Neural Networks 5(2), pp. 241-259, 1992. https://doi.org/10.1016/S0893-6080(05)80023-1